

Template Method

Pin the shape of an algorithm in a base method, and let subclasses fill in the varying steps — without touching the order.

DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

DIVE 02 · 1 km
Train window
Theses, examples, terms

→

DIVE 03 · 100 m
Park walk
How the pieces relate

→

DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

The inheritance twin of Strategy — it freezes an algorithm’s skeleton in a base method and defers only the varying steps to subclasses.

THE CONTOURS

• Invariant order lives once
in the base — variation drops into the gaps

• Hollywood Principle
— don’t call us, the base calls your steps

• Reuse by inheritance,
so it shares the fragile-base-class risk

USE IT WHEN — GoF applicability

One fixed algorithm,
with a few steps that legitimately differ between variants

Common behavior across subclasses
should be factored into one base class to kill duplication

Behavioral family: Strategy · State · Command · Observer · Template Method — it varies steps by subclassing where Strategy varies whole algorithms by composition.

↓ DIVE DEEPER

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

AbstractClass
holds templateMethod() — the fixed skeleton calling primitive & hook operations

primitiveOperation()
an abstract step each subclass must supply; the base only declares it

ConcreteClass
overrides the deferred steps (and optional hooks); never the skeleton

IN THE WILD

java.util.AbstractList
get/size are abstract; iterator/equals are templated on them

HttpServlet.service()
dispatches the request to doGet / doPost hooks you override

JUnit / xUnit
runs setUp() → test → tearDown() around your overrides

React Component
the framework drives the lifecycle, calling render() & hooks

```
<> templateMethod() = step1() → step2() (abstract) → hook() (optional) → step3(), order fixed in the base.
```

GoF intent: “Define the skeleton of an algorithm in an operation, deferring some steps to subclasses.”

↓ DIVE DEEPER

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client
calls templateMethod()

→

AbstractClass
runs the fixed skeleton

→

primitive / hook ?
dispatch to subclass step

→

✓ result
one algorithm, varied behavior

WHY IT FOLLOWS

→ Fixed skeleton → the invariant order is written once and can’t be reordered by subclasses

→ Abstract steps → each subclass plugs only its varying part — duplication stays out of the base

→ Hooks → optional extension points subclasses may override or ignore for safe defaults

STRUCTURE · UML

AbstractClass

```
# config: Options

+ templateMethod(): void
# step1(): void
# step2(): void
# hook(): void
```

templateMethod() is concrete & final-ish: it calls the abstract steps + hook in a fixed order; subclasses override only the steps.

WHAT IT BUYS YOU

✓ Code reuse — the invariant skeleton lives once in the base — the classic factor-out-the-common win

✓ Inverted control — the base calls down into subclass steps, not the other way (Hollywood Principle)

✓ Constrained variation — subclasses can fill steps but cannot change the algorithm’s structure

✓ Defined hook points — optional operations mark exactly where extension is allowed & expected

✓ Inheritance coupling — subclasses bind tightly to the base — fragile-base-class & one-axis-only limits

Factory Method is a Template Method specialized for object creation · Strategy composes the same variation instead of subclassing · hooks resemble Observer extension points.

↓ DIVE DEEPER

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm’s structure.”
— GoF, “Design Patterns”, 1994

THE CODE

```
abstract class Report {
  render(): string { // the template
    return this.header() + this.body();
  }
  protected header(): string { return ""; } // hook
  protected abstract body(): string; // step
}
class Invoice extends Report {
  protected body() { return renderRows(); }
}
```

COMPARE THE ALTERNATIVES

Property	Template	Strategy	Hook fn	HOF
Varies whole algorithm	✗	✓	✗	~
Reuse via inheritance	✓	✗	✗	✗
Swap at runtime	✗	✓	✓	✓
Composes many axes	✗	✓	~	✓
Owns the call order	✓	✗	✗	✓

Best fit → Template: framework lifecycles · Strategy: pluggable policies · Hook/HOF: lightweight one-step variation. (“~” = partial.)

INTERVIEW PITFALLS

Q “Template Method vs Strategy?”
Same goal — vary part of an algorithm. Template uses inheritance & a fixed base skeleton; Strategy composes the varying part as an object, swappable at runtime.

Q “What’s the Hollywood Principle?”
“Don’t call us, we’ll call you” — the base class drives the flow and calls down into your overridden steps; you don’t call the skeleton.

Q “Abstract step vs hook?”
An abstract step is mandatory — the subclass must implement it. A hook has a default (often empty) and override is optional.

Q “Main drawback?”
Inheritance coupling: subclasses are tied to the base, only one variation axis works well, and base changes are fragile.

VARIANTS

Abstract steps
required hooks — subclass must implement, base only declares

Functional template
higher-order fn takes the varying step as a callback — no inheritance

Multiple hooks
several extension points across one algorithm (before/after)

Default hooks
concrete no-op or default; override is optional, not forced

Parameterized
flags / config tweak the skeleton instead of subclassing

vs Strategy
compose the varying step as an object instead of subclassing

NUANCES & GOTCHAS

• Inheritance coupling — subclasses bind to the base’s internals; a base change can break every subclass (fragile base class)

• Abstract vs hook — abstract = required step; hook = optional override with a safe default. Pick deliberately

• Order is fixed — that’s the point: the base owns sequencing, subclasses can’t reorder or skip steps

• Liskov substitution — overridden steps must honor the base’s contract or the templated algorithm misbehaves

• One axis only — two independently-varying steps explode into a subclass matrix; Strategy composes far better

• No real abstract in JS — there are no abstract methods at runtime; throw in the base stub or rely on TS’s abstract

WATCH OUT

⚠ Anti-pattern: Skip it when variation needs to compose along several axes, or change at runtime — that’s a subclass explosion; reach for Strategy/composition. Also avoid forcing inheritance just to share two lines of code.

⚡ Modern take: Prefer a higher-order function or injected step over a base class for simple cases — same fixed skeleton, no inheritance coupling; keep classic Template Method for rich framework lifecycles.

• VasyI Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) v1